



**Instituto Tecnológico de Aeronáutica
Divisão de Engenharia Aeronáutica e Aeroespacial**

Laboratório 5

Redes Adversárias Generativas Profundas (DCGAN) para Geração de Imagens

Deep Learning

Autores:

João Vitor Coelho Guimarães Lima

Professor:

Prof. Marcos Ricardo Omena de Albuquerque Maximo

2 de abril de 2026

1 Introdução

O presente relatório descreve o processo de implementação, treinamento e avaliação de uma **Rede Adversária Generativa Profunda** (*Deep Convolutional Generative Adversarial Network* – **DCGAN**), aplicada à geração de imagens sintéticas de gatos com resolução $3 \times 64 \times 64$ (RGB). Todo o desenvolvimento foi realizado utilizando a *framework* **PyTorch**, em conformidade com as especificações descritas no enunciado do trabalho.

As **Redes Adversárias Generativas** (*Generative Adversarial Networks* – **GANs**), propostas originalmente por Ian Goodfellow, baseiam-se no treinamento simultâneo e competitivo de duas redes neurais: um **Gerador** (G) e um **Discriminador** (D). O Gerador tem por objetivo sintetizar imagens falsas a partir de um vetor de ruído aleatório, enquanto o Discriminador busca distinguir as imagens reais, provenientes do conjunto de dados, daquelas produzidas pelo Gerador. Esse processo configura um **jogo minimax**, no qual as duas redes evoluem ao competir entre si: à medida que o Discriminador se torna mais hábil em detectar falsificações, o Gerador é forçado a produzir amostras cada vez mais realistas.

A **DCGAN** constitui uma variante das GANs na qual ambas as redes são construídas com base em **camadas convolucionais**, empregando convoluções e convoluções transpostas em substituição às camadas totalmente conectadas. Essa arquitetura adota um conjunto de convenções de projeto que estabilizam o treinamento, tais como o uso de *Batch Normalization*, das funções de ativação **ReLU** e **LeakyReLU**, e a ausência de camadas totalmente conectadas nas porções intermediárias das redes.

O objetivo central desta atividade é reproduzir experimentalmente a arquitetura DCGAN, treinando o sistema adversarial de modo a gerar imagens realistas de gatos com dimensão $3 \times 64 \times 64$ e analisando a evolução qualitativa das amostras produzidas, bem como o comportamento das funções de custo ao longo do treinamento.

2 Metodologia

A metodologia adotada neste laboratório consiste na implementação, no treinamento e na avaliação de uma **DCGAN** aplicada ao problema de geração de imagens de gatos. Nas subseções seguintes descrevem-se o conjunto de dados, o espaço latente, as arquiteturas do Gerador e do Discriminador, a estratégia de inicialização dos pesos, as funções de custo e o procedimento de otimização.

2.1 Conjunto de Dados

Foi utilizado um *dataset* composto por fotografias reais de gatos, carregado por meio da classe **ImageFolder** do **torchvision**. As imagens foram redimensionadas para a resolução de 64×64 *pixels* e normalizadas com média $\text{mean} = [0,5, 0,5, 0,5]$ e desvio padrão $\text{std} = [0,5, 0,5, 0,5]$ para cada canal de cor. Essa normalização leva os valores dos *pixels* ao intervalo $[-1, 1]$, tornando-os compatíveis com a função de ativação **Tanh** empregada na camada de saída do Gerador. O carregamento dos dados foi realizado em lotes de tamanho `batch_size = 128`.

2.2 Espaço Latente

O Gerador recebe como entrada um vetor de ruído aleatório, denominado **vetor latente**, de dimensão `latent_size` = 100. Esse vetor é amostrado de uma **distribuição normal** e organizado no formato (batch, 100, 1, 1), adequado para alimentar a primeira camada de convolução transposta. O espaço latente funciona como a fonte de variabilidade do modelo: diferentes amostras de ruído conduzem a diferentes imagens sintéticas na saída do Gerador.

2.3 Arquitetura do Gerador

O **Gerador** é responsável por transformar o vetor latente de dimensão 100 em uma imagem RGB de dimensão $3 \times 64 \times 64$. Para isso, utiliza uma sequência de cinco camadas de **convolução transposta** (`ConvTranspose2d`), parametrizadas pelo multiplicador de tamanho `size_multiplier` = 64, que aumentam progressivamente a resolução espacial enquanto reduzem o número de canais, conforme apresentado na Tabela 1.

Tabela 1: Arquitetura do Gerador da DCGAN.

Camada	Tipo	Canais	Saída	Kernel	Stride	Ativação
Entrada	Ruído	100	1×1	—	—	—
1	ConvTranspose2d	512	4×4	4×4	1	ReLU
2	ConvTranspose2d	256	8×8	4×4	2	ReLU
3	ConvTranspose2d	128	16×16	4×4	2	ReLU
4	ConvTranspose2d	64	32×32	4×4	2	ReLU
5	ConvTranspose2d	3	64×64	4×4	2	Tanh

Cada bloco intermediário é composto pela sequência `ConvTranspose2d` + `BatchNorm2d` + `ReLU`. A primeira camada utiliza `kernel` = 4, `stride` = 1 e `padding` = 0, expandindo o vetor latente de 1×1 para um mapa de características 4×4 . As camadas seguintes utilizam `kernel` = 4, `stride` = 2 e `padding` = 1. A última camada produz 3 canais e emprega a ativação **Tanh**, garantindo que a saída esteja no intervalo $[-1, 1]$, em consonância com a normalização aplicada ao conjunto de dados.

A dimensão espacial da saída de uma convolução transposta é dada pela Equação 1:

$$\text{saída} = (\text{entrada} - 1) \cdot \text{stride} - 2 \cdot \text{padding} + \text{kernel}. \quad (1)$$

A partir da Equação 1, observa-se que o uso de `stride` = 2, com `kernel` = 4 e `padding` = 1, dobra a resolução espacial a cada camada, conduzindo à sucessão $4 \times 4 \rightarrow 8 \times 8 \rightarrow 16 \times 16 \rightarrow 32 \times 32 \rightarrow 64 \times 64$.

2.4 Arquitetura do Discriminador

O **Discriminador** é, essencialmente, o espelho do Gerador. Ele recebe uma imagem RGB de dimensão $3 \times 64 \times 64$ e produz uma única probabilidade escalar, no intervalo $[0, 1]$, que representa a confiança de que a imagem seja real. A rede é composta por cinco camadas de **convolução** (`Conv2d`) que reduzem progressivamente a resolução espacial enquanto aumentam o número de canais, conforme a Tabela 2.

Tabela 2: Arquitetura do Discriminador da DCGAN.

Camada	Tipo	Canais	Saída	Kernel	Stride	Ativação
Entrada	Imagem	3	64×64	—	—	—
1	Conv2d	64	32×32	4×4	2	LeakyReLU(0,2)
2	Conv2d	128	16×16	4×4	2	LeakyReLU(0,2)
3	Conv2d	256	8×8	4×4	2	LeakyReLU(0,2)
4	Conv2d	512	4×4	4×4	2	LeakyReLU(0,2)
5	Conv2d	1	1×1	4×4	1	Sigmoid

Cada bloco é composto pela sequência **Conv2d** + **BatchNorm2d** + **LeakyReLU(0,2)**. A camada final aplica uma operação de *Flatten* seguida da função **Sigmoid**, produzindo a probabilidade de a imagem ser real. A dimensão espacial da saída de uma convolução é dada pela Equação 2:

$$\text{saída} = \left\lfloor \frac{\text{entrada} + 2 \cdot \text{padding} - \text{kernel}}{\text{stride}} \right\rfloor + 1. \quad (2)$$

O emprego da função **LeakyReLU** no Discriminador justifica-se por sua capacidade de mitigar o problema dos **neurônios mortos**: ao permitir um pequeno gradiente para entradas negativas (com inclinação 0,2), a função mantém o fluxo de gradiente durante a retropropagação, contribuindo para a estabilidade do treinamento adversarial. Já a função **Sigmoid** é adotada na saída por produzir um valor no intervalo $[0, 1]$, diretamente interpretável como uma **probabilidade**.

2.5 Inicialização dos Pesos

Seguindo a convenção estabelecida para DCGANs, os pesos das redes foram inicializados de forma específica. Os pesos das camadas convolucionais (**Conv2d** e **ConvTranspose2d**) foram amostrados de uma distribuição normal com média 0,0 e desvio padrão 0,02. Já os pesos das camadas de *Batch Normalization* (parâmetro γ) foram amostrados de uma distribuição normal com média 1,0 e desvio padrão 0,02, com o respectivo *bias* (β) inicializado em 0.

A escolha da **média 0** para os *kernels* convolucionais decorre do fato de seus coeficientes serem somados nas operações de convolução; iniciá-los pequenos e centrados em zero evita saturações e explosões de ativação no início do treinamento. Por outro lado, a **média 1** adotada para o fator multiplicativo γ da *Batch Normalization* faz com que essa camada comece operando aproximadamente como uma transformação **identidade**, preservando a escala das ativações normalizadas até que o treinamento ajuste tais parâmetros.

2.6 Função de Custo

A função de custo empregada é a **Entropia Cruzada Binária** (*Binary Cross-Entropy* – **BCE**). O treinamento adversarial pode ser formalizado como o problema de otimização **minimax** descrito na Equação 3:

$$\min_G \max_D V(D, G) = \mathbb{E}_x [\log D(x)] + \mathbb{E}_z [\log (1 - D(G(z)))] . \quad (3)$$

Na prática, as perdas das duas redes são calculadas separadamente a cada passo de treinamento. Para o **Discriminador**, as imagens reais são rotuladas com alvo 1 e as imagens falsas (geradas) com alvo 0, de modo que a perda total é dada pela Equação 4:

$$\mathcal{L}_D = \text{BCE}(D(\text{reais}), 1) + \text{BCE}(D(\text{falsas}), 0). \quad (4)$$

Durante o cálculo da perda do Discriminador, aplica-se a operação `detach()` sobre as imagens falsas, de modo a impedir que o gradiente seja propagado de volta ao Gerador nessa etapa. Para o **Gerador**, cujo objetivo é **enganar** o Discriminador, as imagens geradas recebem alvo 1, resultando na perda da Equação 5:

$$\mathcal{L}_G = \text{BCE}(D(G(z)), 1). \quad (5)$$

2.7 Otimização e Treinamento

A otimização de ambas as redes foi realizada com o algoritmo **Adam**, configurado com taxa de aprendizado `learning_rate` = 0,0002 e parâmetros de momento $\beta_1 = 0,5$ e $\beta_2 = 0,999$, valores recomendados pela literatura de DCGANs por favorecerem a estabilidade do treinamento adversarial. A cada passo de treinamento (`fit_step`), o **Discriminador** é atualizado primeiramente, seguido da atualização do **Gerador**. O processo foi conduzido por `num_epochs` = 50 épocas.

3 Resultados

O desempenho do modelo DCGAN foi avaliado com base na evolução qualitativa das imagens geradas ao longo das épocas e nas métricas quantitativas de perda e *score*, conforme detalhado a seguir.

A Figura 1 apresenta as imagens produzidas pelo Gerador ao final da primeira época de treinamento. Nesse estágio inicial, as amostras ainda se assemelham a **ruído aleatório**, sem estrutura espacial discernível, refletindo o fato de que os pesos do Gerador ainda estão próximos de sua inicialização e não capturaram os padrões característicos das imagens reais.

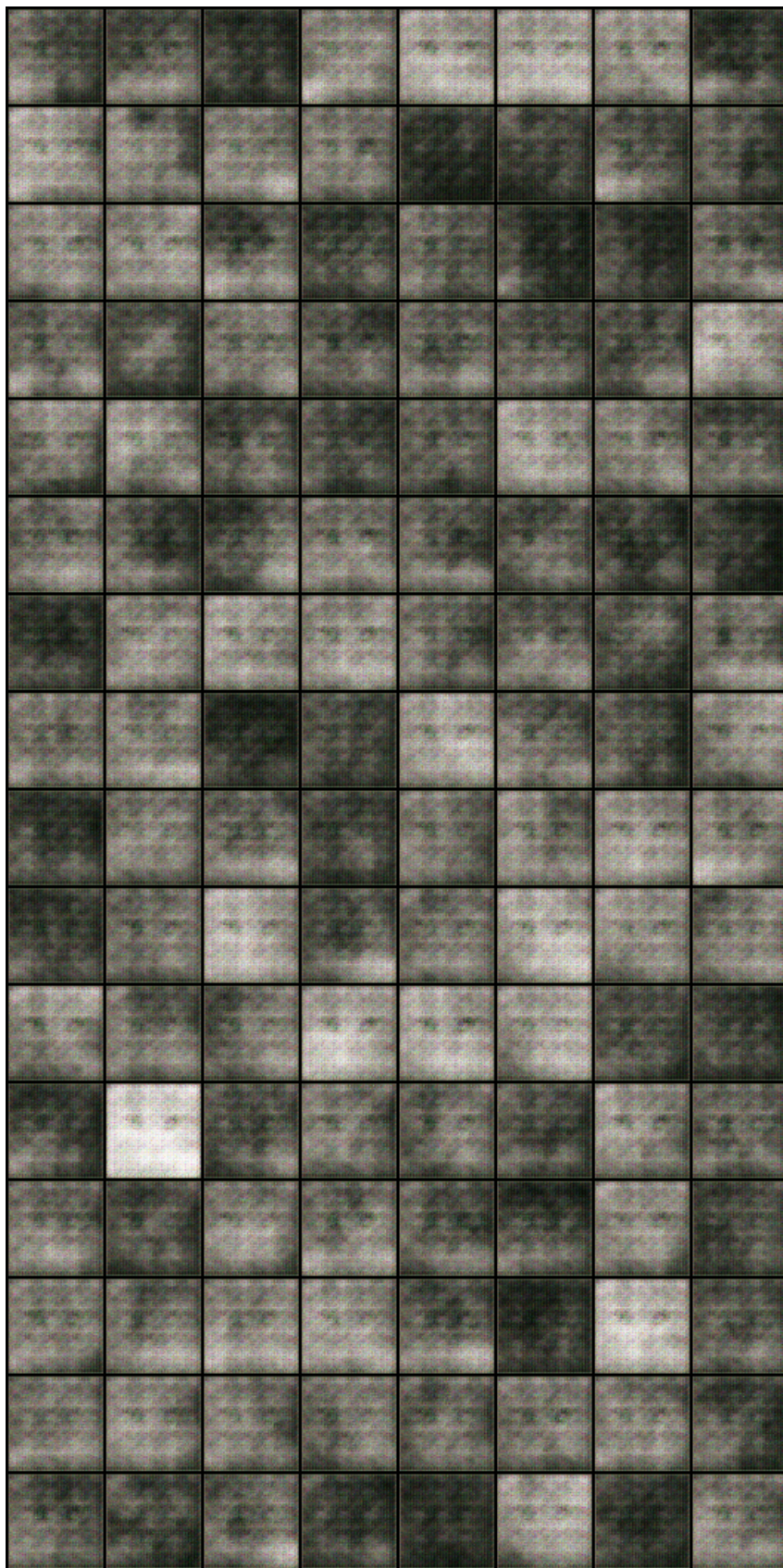


Figura 1: Imagens geradas na 1ª época — ainda próximas de ruído aleatório.

A Figura 2 mostra as imagens geradas na décima época. Já se notam as primeiras manchas de cor coerentes e o surgimento de regiões com contraste, indicando que o Gera-

dor começou a abandonar o regime puramente ruidoso e a organizar a distribuição espacial dos *pixels*.

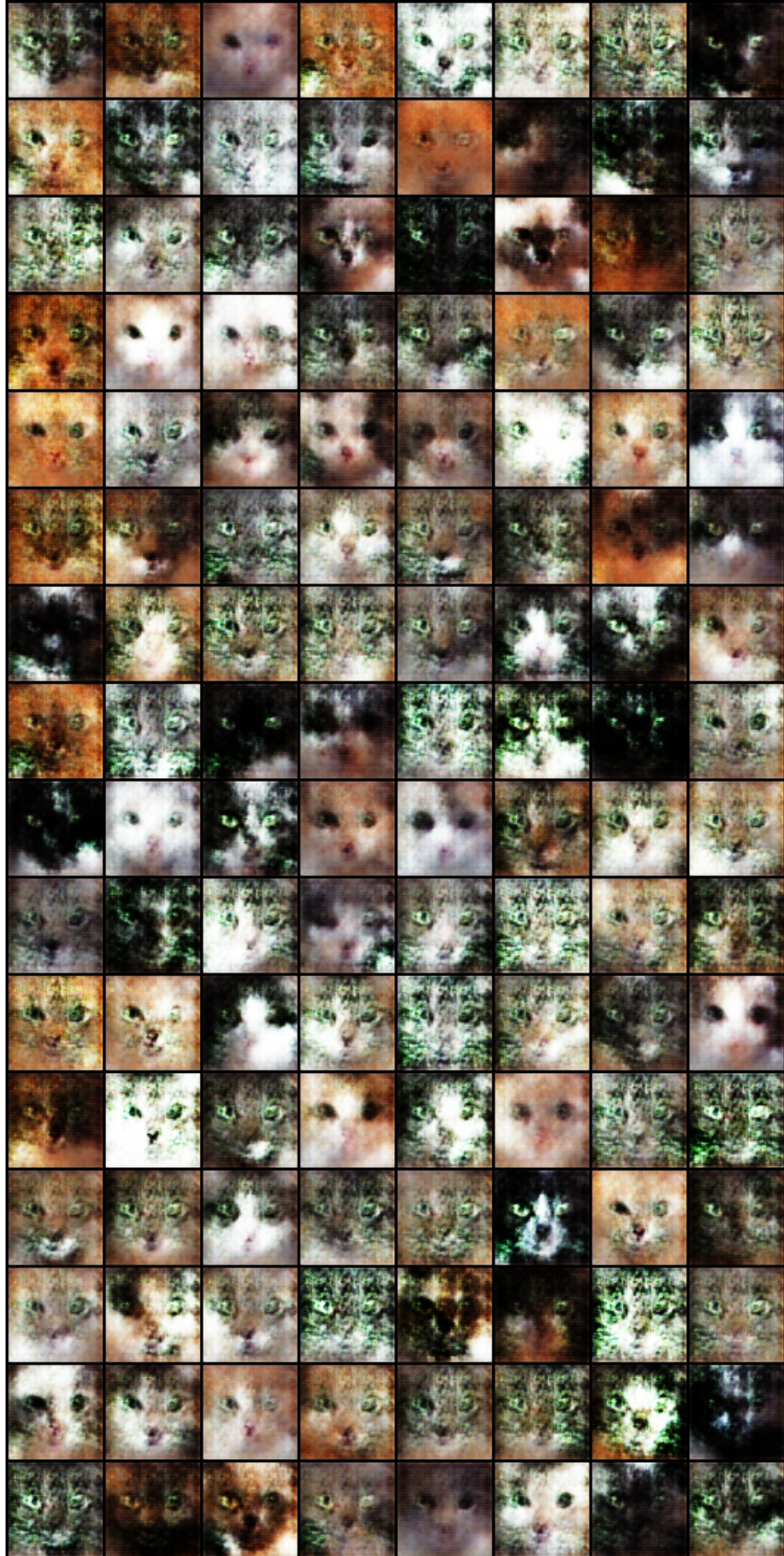


Figura 2: Imagens geradas na 10ª época — surgimento de regiões coerentes de cor.

A Figura 3 ilustra as imagens geradas na vigésima quinta época. Observa-se que, nesse ponto intermediário do treinamento, **estruturas de gatos começam a emergir**: tornam-se perceptíveis contornos, regiões de textura semelhantes a pelagem e disposições espaciais que sugerem faces de animais. Esse comportamento evidencia que o Gerador passou a internalizar características relevantes do conjunto de dados ao longo das épocas.



Figura 3: Imagens geradas na 25ª época — estruturas de gatos começam a emergir.

A Figura 4 apresenta as amostras obtidas na quadragésima época, nas quais as formas de gatos tornam-se mais nítidas e consistentes, com olhos, orelhas e texturas de pelagem

mais bem definidos em relação aos estágios anteriores.

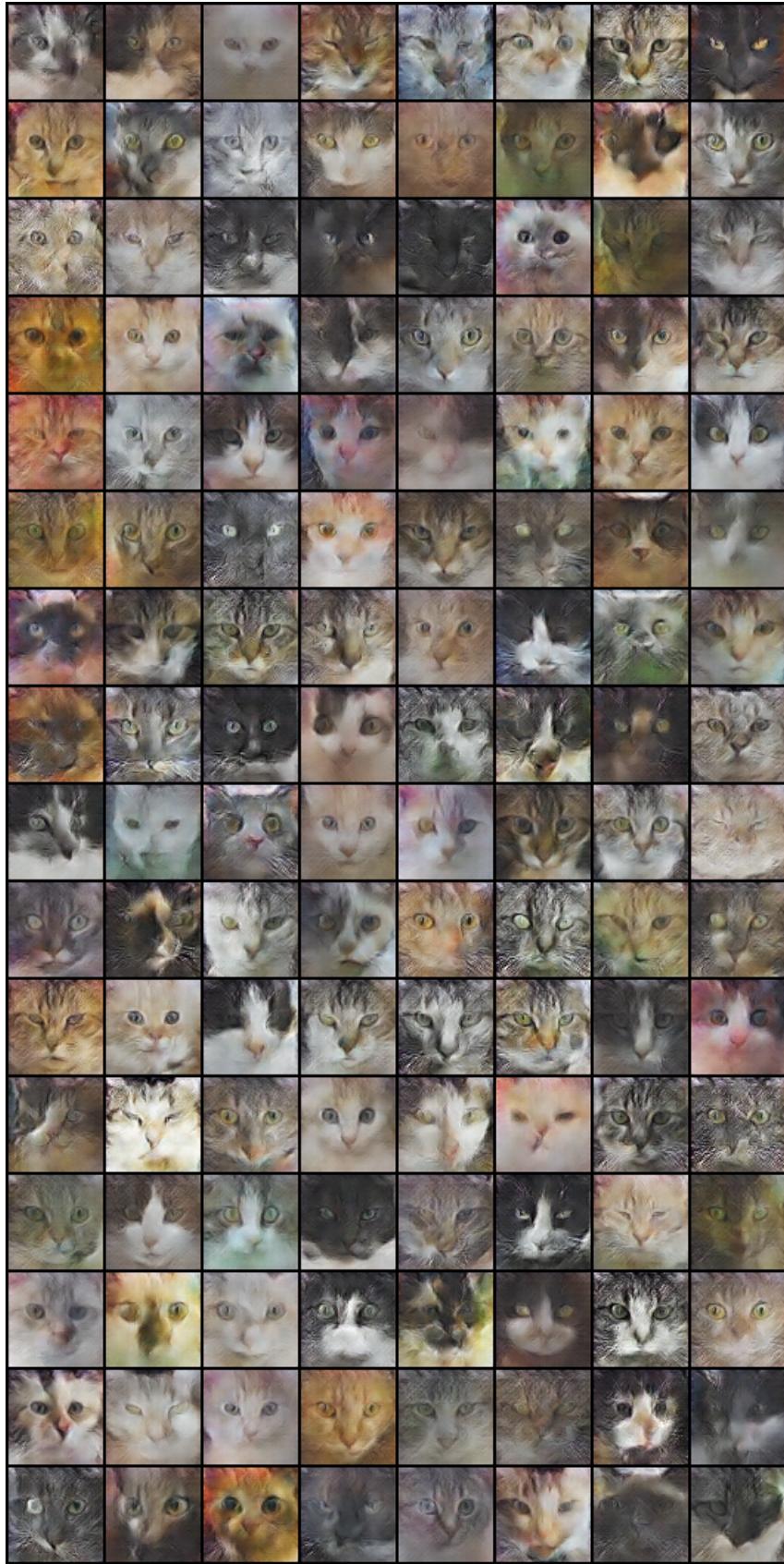


Figura 4: Imagens geradas na 40ª época — formas mais nítidas e consistentes.

A Figura 5 apresenta o resultado final do Gerador, obtido ao término da quinquagésima época. As imagens exibem maior coerência e nível de detalhe, com formas de gatos mais bem definidas, embora ainda apresentem imperfeições e artefatos típicos de modelos generativos treinados por um número limitado de épocas e com volume restrito de dados.



Figura 5: Imagens geradas na 50ª (última) época — resultado final do gerador.

A análise conjunta das Figuras 1, 2, 3, 4 e 5 demonstra uma **melhora visual progressiva** das amostras geradas, partindo de ruído puro e evoluindo gradualmente para

imagens reconhecíveis. Essa progressão é o reflexo direto da dinâmica adversarial, na qual o aprimoramento contínuo do Discriminador pressiona o Gerador a produzir falsificações cada vez mais convincentes.

A Figura 6 apresenta a evolução das funções de perda do Gerador e do Discriminador ao longo das épocas. Diferentemente de problemas de aprendizado supervisionado convencionais, em que se espera uma queda monotônica da perda, o treinamento de GANs exibe um comportamento **oscilatório**: as duas curvas competem entre si, de modo que a diminuição da perda de uma rede frequentemente corresponde ao aumento da perda da outra. Idealmente, o sistema busca um **equilíbrio** dinâmico, no qual nenhuma das redes domina completamente a outra.

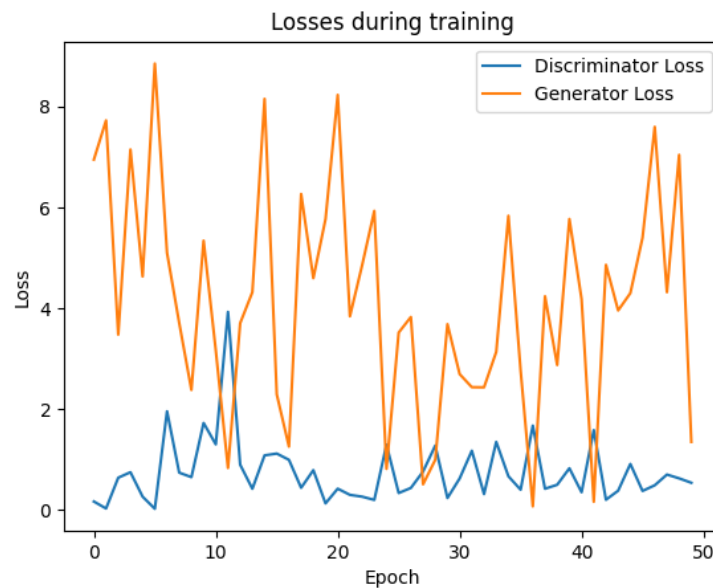


Figura 6: Evolução das perdas (*loss*) do gerador e do discriminador ao longo das épocas.

A Figura 7 apresenta a evolução dos *scores* médios atribuídos pelo Discriminador às imagens reais (**real_score**) e às imagens falsas (**fake_score**). Espera-se que o **real_score** se mantenha **elevado**, indicando que o Discriminador reconhece corretamente as imagens reais, enquanto o **fake_score** tende a **aumentar** à medida que o Gerador melhora e produz amostras mais realistas. Em um cenário de equilíbrio ideal, ambos os *scores* convergem para valores próximos de 0,5, situação em que o Discriminador não consegue mais distinguir, com confiança, imagens reais de imagens geradas.

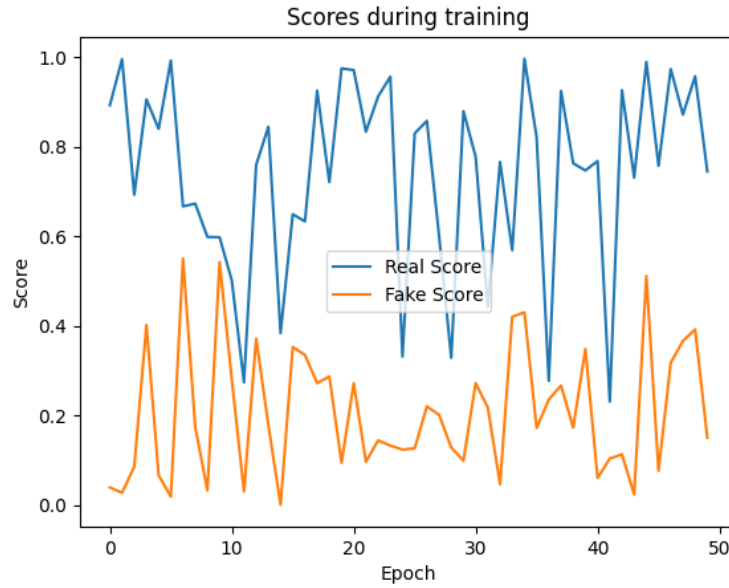


Figura 7: Evolução dos *scores* médios atribuídos pelo discriminador às imagens reais e falsas.

De maneira geral, conclui-se que o comportamento observado tanto nas curvas de perda quanto nos *scores* é consistente com a dinâmica esperada de uma DCGAN, evidenciando a competição equilibrada entre as duas redes e a melhora gradual da qualidade das amostras sintetizadas.

4 Conclusão

Neste laboratório, implementou-se e treinou-se uma **Rede Adversária Generativa Profunda** (DCGAN) em **PyTorch**, com o objetivo de gerar imagens sintéticas de gatos com resolução $3 \times 64 \times 64$. Foram desenvolvidas as arquiteturas do **Gerador**, baseado em camadas de convolução transposta, e do **Discriminador**, espelho convolucional do primeiro, ambas seguindo as convenções de projeto da DCGAN, incluindo o uso de *Batch Normalization*, das ativações **ReLU**, **LeakyReLU** e **Tanh/Sigmoid**, e da estratégia específica de **inicialização dos pesos**.

O treinamento foi conduzido de forma **adversarial**, utilizando a função de custo de **Entropia Cruzada Binária** e o otimizador **Adam**, com atualização sequencial do Discriminador e do Gerador a cada passo. Os resultados evidenciaram uma melhora qualitativa progressiva das imagens geradas, partindo de ruído aleatório na primeira época e evoluindo para formas de gatos reconhecíveis ao final das 50 épocas de treinamento.

Cabe destacar as **dificuldades típicas** associadas ao treinamento de GANs, sobretudo a **instabilidade** inerente ao processo adversarial e a necessidade de **balancear** o aprendizado das duas redes, de modo a evitar que uma supere a outra de forma prematura. O comportamento oscilatório das perdas e a convergência dos *scores* ilustram bem esse delicado equilíbrio.

Como **possíveis melhorias**, sugere-se o treinamento por um número maior de épocas, a ampliação e diversificação do conjunto de dados, bem como a adoção de técnicas mais modernas de geração de imagens, tais como variantes mais estáveis de GANs ou métodos baseados em difusão, capazes de produzir amostras de maior qualidade e fidelidade.